CMSC433 / Part 04

Monad tracing

Guided practice

Use with the chat: predict one part before receiving feedback.

Idea. Maybe bind propagates failure. List bind enumerates each earlier choice fully before advancing to the next. `return` does not exit a do block.

Worked example. `do x <- [1,3 :: Int]; return (x+2)` yields `[3,5]`. There is one result per successful branch.

Use this supplied collector:

```
gather :: Monad m => [m a] -> m [a]
gather [] = return []
gather (u:us) = do
  x <- u
  xs <- gather us
  return (x:xs)
```

1. Maybe execution

Evaluate `gather [Just 'a', Nothing]`; give the whole result type.

Answer: [Value, type, and where execution stops producing results.]

2. Enumerate choices

Evaluate `gather [[True,False], [True,False]]`, in order.

Answer: [One row per outer choice; final result.]

3. Empty input versus empty action

Evaluate both `gather ([] :: [[Bool]])` and `gather [[] :: [Bool]]`. Explain why their results differ.

Answer: [Both values and explanation.]

Source: Sample Q4, p. 4; Monads List Monad section; GenericMonads collector.